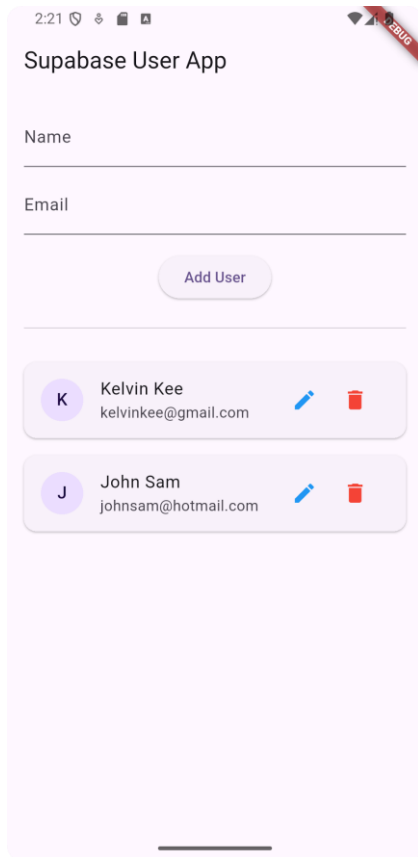
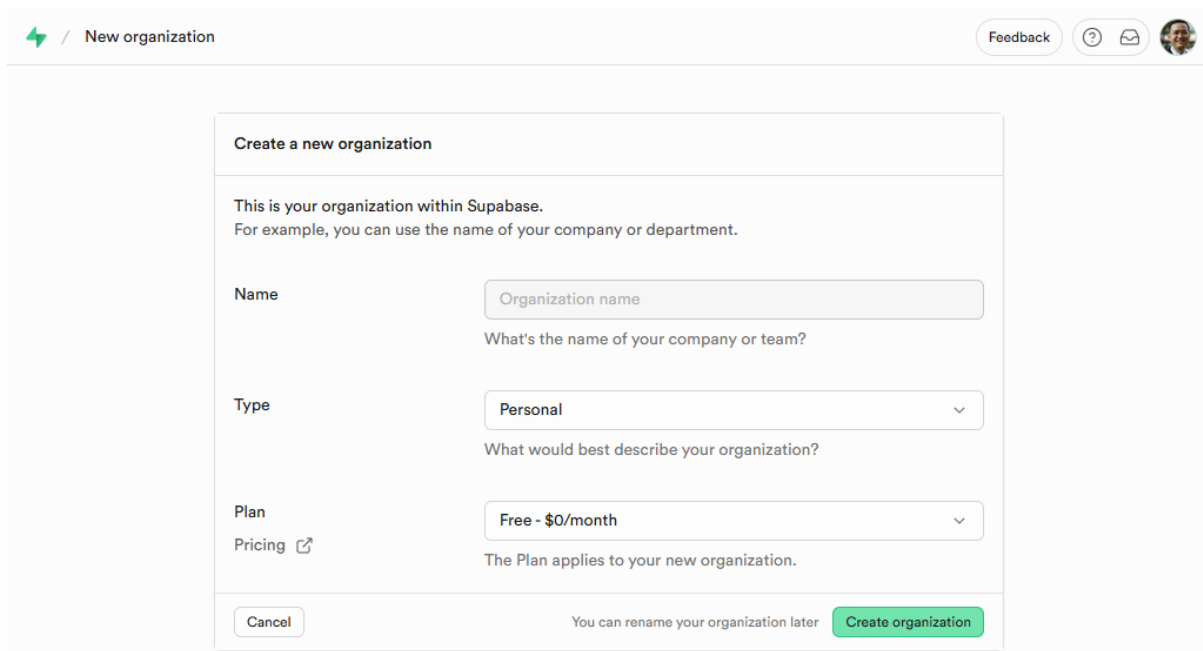


Practical 11: Supabase – Backend as a Service (BaaS)

Supabase, an alternative to Firebase, provides a full Postgres database for every project with Realtime functionality, database backups, extensions, and more. This lab demonstrates the processes of creating a Flutter app that can perform CRUD of contact records as shown below:



1. Visit <https://supabase.com/>, register a new organisation. Select a type and opt for the **Free** plan. Press the **Create organization** button to proceed.



- Next, assign a **project name** and select a **region**, e.g. Southeast Asia (Singapore). Press the **Create project** button to continue.

Create a new project
Your project will have its own dedicated instance and full Postgres database. An API will be set up so you can easily interact with your new database.

Organization: BMIT2073 Free

Project name: seekweateck's Project

Database Password: Type in a strong password
This is the password to your Postgres database, so it must be strong and hard to guess. [Generate a password](#)

Region: Southeast Asia (Singapore)
Select the region closest to your users for the best performance.

- Once your project has been created, you will be prompted to the dashboard of your project. Scroll to the bottom to view main features offered by Supabase.

seekweateck's Project NANO

Tables: 0 Functions: 0 Replicas: 0 Project Status: ● Project Status

Welcome to your new project
Your project has been deployed on its own instance, with its own API all set up and ready to use.

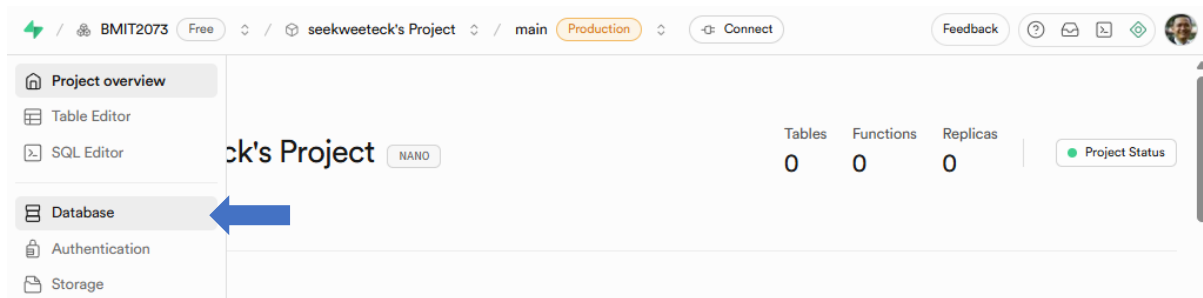
Get started by building out your database
Start building your app by creating tables and inserting data. Our Table Editor makes Postgres as easy to use as a spreadsheet, but there's also our SQL Editor if you need something more.

Table Editor SQL Editor About Database

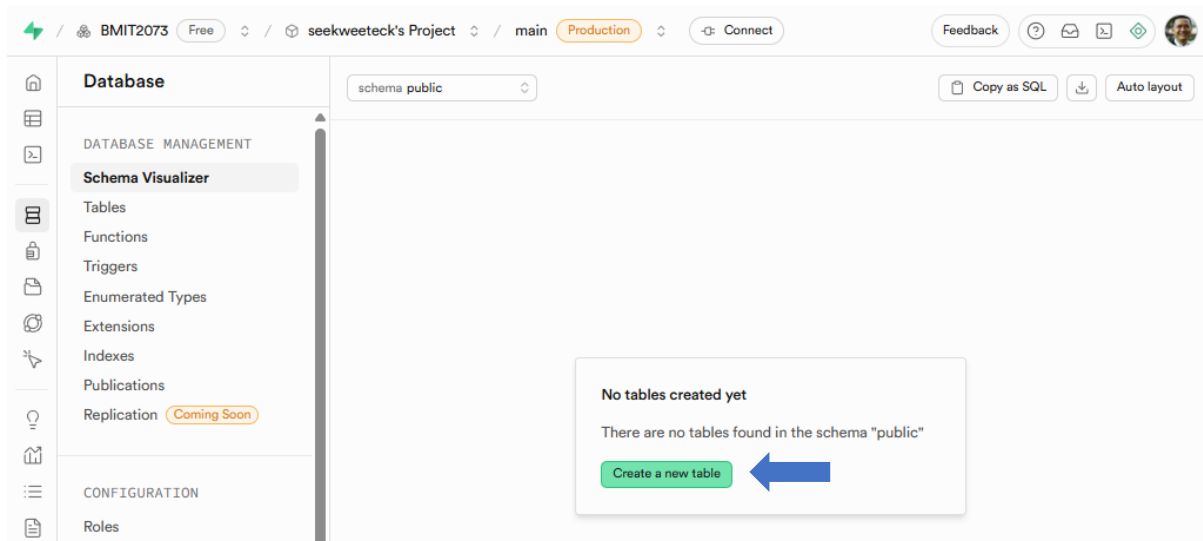
	id	task	status
1	1	Create a project	Complete
2	2	Read documentation	Complete
3	3	Build application	In progress
4	4	Connect Supabase	In progress
5	5	Deploy project	Not started
6	6	Get users	Not started
7	7	Upgrade to Pro	Not started

Some of the core features are PostgreSQL database, authentication, Realtime engine, storage, etc. Visit <https://supabase.com/docs/guides/getting-started/features> to find out more.

4. Next, let us create a new database. Click on the left panel and select **Database**.



5. Click the **Create a new table** button to continue.



6. Enter a table **name** and scroll down to the bottom.

The form is titled 'Create a new table under public'. It has a 'Name' field with a cursor and a 'Description' field with the placeholder text 'Optional'.

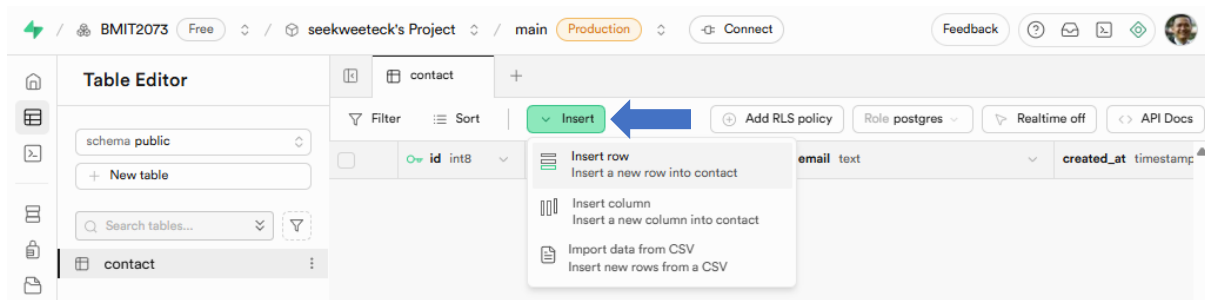
7. Add two additional columns; **name** and **email**. Click the **Save** button to complete the task.

The 'Columns' table shows the following columns:

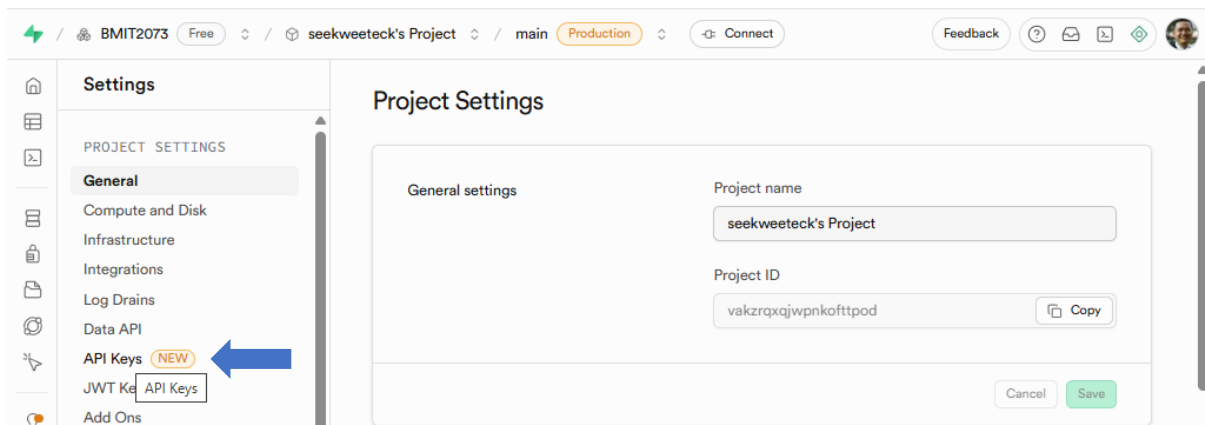
Name	Type	Default Value	Primary
id	# int8	NULL	☒
name	T varchar	NULL	☐
email	T text	NULL	☐
created_at	timestamp	now()	☐

At the bottom, there is an 'Add column' button.

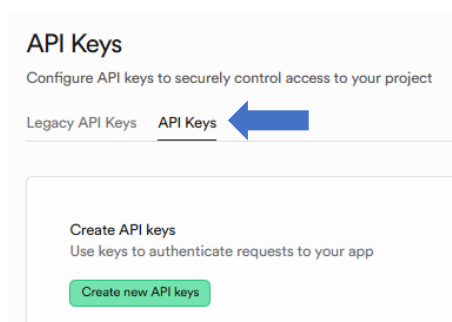
8. On the top panel, click the **Insert -> Insert row** to insert some records into the **contact** table.



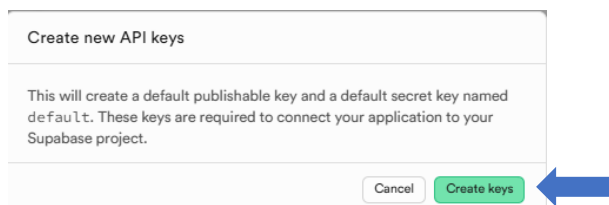
9. Next, we will enable the backend API key, which will allow applications to access the database. On the left panel, click **Project Settings** ( **Project Settings**) and then click **API Keys**.



10. Next, click the **API Keys** tab and then the **Create new API keys** button.

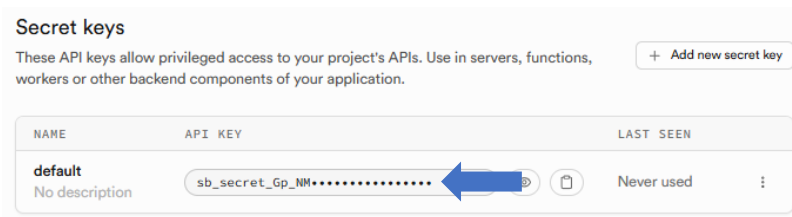


Click **Create keys** when prompt.

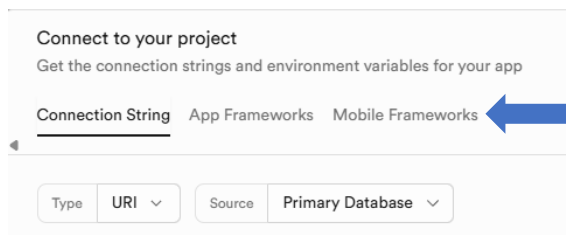


11. Next, click the **API Keys** tab and then the **Create new API keys** button.

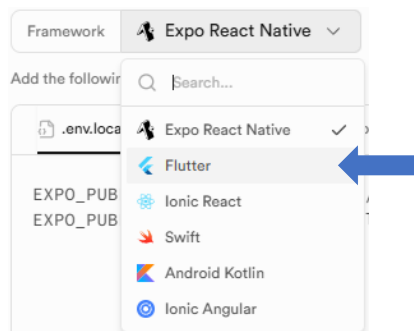
12. Copy and paste the **Secret keys** to a text file. You will need it later in your Flutter app.



13. On the top panel of the dashboard, click the **Connect** () button, and then click the **Mobile Frameworks** tab.



14. In the Framework dropdown list, select **Flutter**.



15. Copy and paste the **server URL** to a text file.



16. Next, let us create a new Flutter mobile app. Start **Android Studio**, and create a new Flutter project using the **Empty** template.

17. Once your project is ready, open a new terminal window, enter the following command to insert the Supabase dependencies:

```
flutter pub add supabase_flutter
```

18. Enter the following codes. Also, copy and paste the **server URL** and **Secret keys** to the program code.

```

import 'package:flutter/material.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

const String supabaseUrl = 'SERVER URL';
const String supabaseKey = 'API KEY';

class User {
  final String id;
  final String name;
  final String email;

  User({required this.id, required this.name, required this.email});

  factory User.fromJson(Map<String, dynamic> json) {
    return User(
      id: json['id'].toString(), // Supabase 'id' field is an integer/bigint
      name: json['name'] as String,
      email: json['email'] as String,
    );
  }
}

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Supabase.initialize(
    url: supabaseUrl,
    anonKey: supabaseKey,
  );
  runApp(MyApp());
}

final supabase = Supabase.instance.client;

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Supabase',
      theme: ThemeData(

        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
      ),
      home: const UserListPage(),
    );
  }
}

class UserListPage extends StatefulWidget {
  const UserListPage({super.key});

  @override
  State<UserListPage> createState() => _UserListPageState();
}

class _UserListPageState extends State<UserListPage> {
  // Supabase client instance
  final supabase = Supabase.instance.client;

```

```

// List to hold the fetched users
List<User> _users = [];
bool _isLoading = true;

// Text controllers for input fields
final _nameController = TextEditingController();
final _emailController = TextEditingController();

// A selected user for updating or deleting
User? _selectedUser;

@override
void initState() {
  super.initState();
  _fetchUsers();
}

@override
void dispose() {
  _nameController.dispose();
  _emailController.dispose();
  super.dispose();
}

// Fetches all users from the 'user' table
Future<void> _fetchUsers() async {
  setState(() {
    _isLoading = true;
  });

  try {
    // Use select() to get all rows from the 'user' table.
    final response = await supabase.from('contact').select();

    // Parse the list of JSON objects into a list of User objects.
    final users = (response as List).map((item) => User.fromJson(item)).toList();

    setState(() {
      _users = users;
    });
  } catch (e) {
    // Handle any errors during the fetch
    print('Error fetching users: $e');
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Failed to fetch users: $e')),
    );
  } finally {
    setState(() {
      _isLoading = false;
    });
  }
}

// Adds a new user to the 'user' table
Future<void> _addUser() async {
  final name = _nameController.text.trim();
  final email = _emailController.text.trim();

  if (name.isEmpty || email.isEmpty) {
    return;
  }

```

```

    }

    try {
        setState(() {
            _isLoading = true;
        });
        // Use insert() to add a new row to the 'user' table.
        final response = await supabase.from('contact').insert({
            'name': name,
            'email': email,
        }).select();

        _nameController.clear();
        _emailController.clear();

        // Since we used .select(), Supabase returns the inserted row.
        // We can add it directly to our list to avoid re-fetching all data.
        final newUser = User.fromJson((response as List).first);
        setState(() {
            _users.add(newUser);
        });

        ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('User added successfully!')),
        );
    } catch (e) {
        print('Error adding user: $e');
        ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text('Failed to add user: $e')),
        );
    } finally {
        setState(() {
            _isLoading = false;
        });
    }
}

// Updates an existing user in the 'user' table
Future<void> _updateUser() async {
    if (_selectedUser == null) {
        return;
    }

    final name = _nameController.text.trim();
    final email = _emailController.text.trim();

    if (name.isEmpty && email.isEmpty) {
        return;
    }

    try {
        setState(() {
            _isLoading = true;
        });
        // Use update() and eq() to target a specific row by its 'id'.
        final response = await supabase.from('contact').update({
            'name': name.isNotEmpty ? name : _selectedUser!.name,
            'email': email.isNotEmpty ? email : _selectedUser!.email,
        }).eq('id', _selectedUser!.id).select();

        _nameController.clear();

```



```

_emailController.clear();

// Find and update the user in our local list.
final updatedUser = User.fromJson((response as List).first);
final index = _users.indexWhere((u) => u.id == updatedUser.id);
if (index != -1) {
  setState(() {
    _users[index] = updatedUser;
  });
}

ScaffoldMessenger.of(context).showSnackBar(
  const SnackBar(content: Text('User updated successfully!')),
);
} catch (e) {
  print('Error updating user: $e');
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('Failed to update user: $e')),
  );
} finally {
  setState(() {
    _isLoading = false;
    _selectedUser = null;
  });
}
}

// Deletes a user from the 'user' table
Future<void> _deleteUser(String userId) async {
  try {
    setState(() {
      _isLoading = true;
    });
    // Use delete() and eq() to remove a specific row.
    await supabase.from('contact').delete().eq('id', userId);

    // Remove the user from our local list to update the UI.
    setState(() {
      _users.removeWhere((user) => user.id == userId);
    });

    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('User deleted successfully!')),
    );
  } catch (e) {
    print('Error deleting user: $e');
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Failed to delete user: $e')),
    );
  } finally {
    setState(() {
      _isLoading = false;
    });
  }
}

// Fills the text fields with the selected user's data for editing
void _selectUserForUpdate(User user) {
  setState(() {
    _selectedUser = user;
    _nameController.text = user.name;
  });
}

```

```

        _emailController.text = user.email;
    });
}

// Clears the text fields and selection
void _clearSelection() {
    setState(() {
        _selectedUser = null;
        _nameController.clear();
        _emailController.clear();
    });
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(
            title: const Text('Supabase User App'),
        ),
        body: Padding(
            padding: const EdgeInsets.all(16.0),
            child: Column(
                children: [
                    // Input fields for name and email
                    TextField(
                        controller: _nameController,
                        decoration: const InputDecoration(labelText: 'Name'),
                    ),
                    const SizedBox(height: 8),
                    TextField(
                        controller: _emailController,
                        decoration: const InputDecoration(labelText: 'Email'),
                    ),
                    const SizedBox(height: 16),
                    // Action buttons
                    Row(
                        mainAxisAlignment: MainAxisAlignment.spaceEvenly,
                        children: [
                            ElevatedButton(
                                onPressed: _selectedUser != null ? _updateUser : _addUser,
                                child: Text(_selectedUser != null ? 'Update User' : 'Add User'),
                            ),
                            if (_selectedUser != null)
                                TextButton(
                                    onPressed: _clearSelection,
                                    child: const Text('Cancel'),
                                ),
                        ],
                    ),
                    const SizedBox(height: 16),
                    const Divider(),
                    const SizedBox(height: 16),
                    // User list view
                    Expanded(
                        child: _isLoading
                            ? const Center(child: CircularProgressIndicator())
                            : _users.isEmpty
                                ? const Center(child: Text('No users found. Add one!'))
                                : ListView.builder(
                                    itemCount: _users.length,
                                    itemBuilder: (context, index) {

```

```
final user = _users[index];  
return Card(  
    margin: const EdgeInsets.symmetric(vertical: 8),  
    child: ListTile(  
        leading: CircleAvatar(child: Text(user.name[0])),  
        title: Text(user.name),  
        subtitle: Text(user.email),  
        trailing: Row(  
            mainAxisAlignment: MainAxisAlignment.min,  
            children: [  
                IconButton(  
                    icon: const Icon(Icons.edit, color: Colors.blue),  
                    onPressed: () => _selectUserForUpdate(user),  
                ),  
                IconButton(  
                    icon: const Icon(Icons.delete, color: Colors.red),  
                    onPressed: () => _deleteUser(user.id),  
                ),  
            ],  
        ),  
    ),  
);
```

That's all for this practical!



TODO: Visit <https://supabase.com/docs/guides/getting-started/tutorials/with-flutter> to learn more about Supabase.

Find out how Supabase can do the followings: -

- Utilise advanced database features such as stored procedure, trigger, functions etc.
- Custom Python Backend API